Arquitectura técnica y guardado

Oct 21, 2026 · @andoni

1. Resumen

Golden Moments FC se construye en cuatro ensamblados, con toda la lógica del juego fuera de Unity, una base de datos SQLite por partida y guardados comprimidos con zstd. Este documento fija cómo se cumplen las siete decisiones confirmadas y corrige tres detalles técnicos que no funcionarían tal cual (marcados como **Ajuste**).

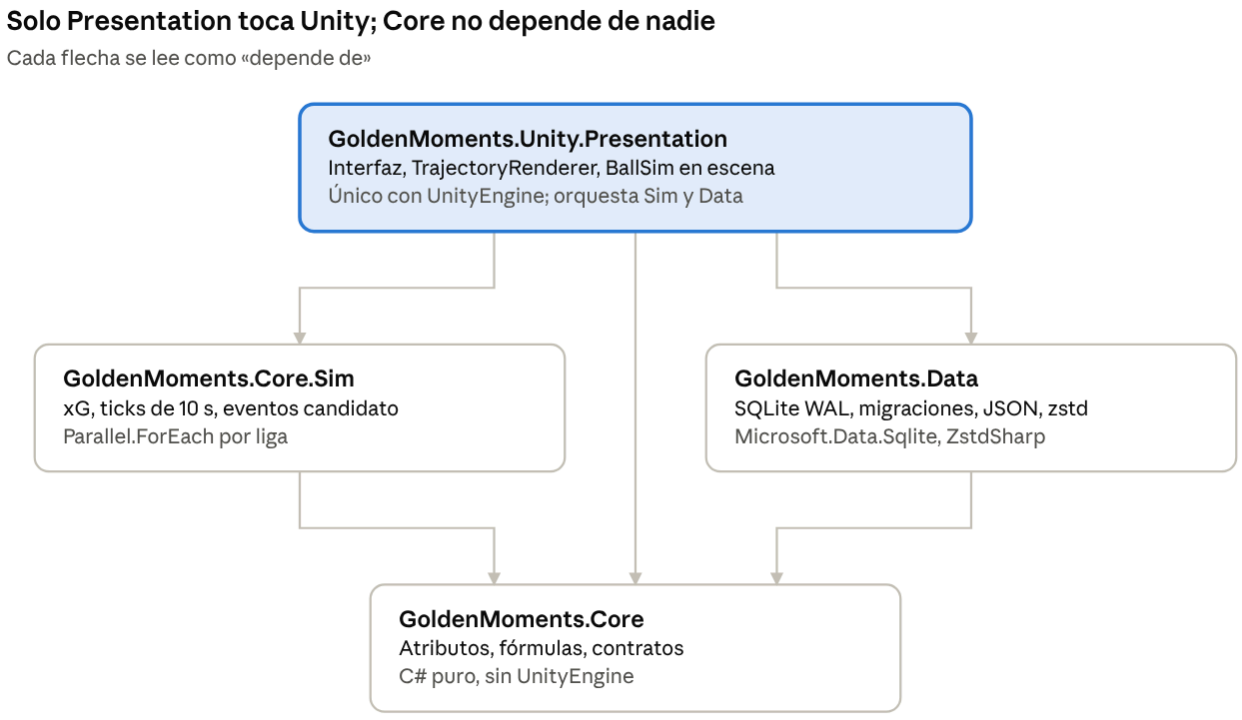
Área	Decisión	Estado
Ensamblados	Core, Core.Sim, Data y Unity.Presentation; solo Presentation ve `UnityEngine`	Confirmado
Base de datos	Un `.sqlite` por partida, `Microsoft.Data.Sqlite`, modo WAL, migraciones numeradas	Confirmado
Guardado	Copia del `.sqlite` comprimida con zstd nivel 3, menos de 150 MB a 20 temporadas	Confirmado, con ajuste en cómo se copia
Determinismo	IEEE 754 con semillas explícitas; `BallSim` a 120 Hz con paso fijo	Confirmado, con ajuste en funciones matemáticas
Repeticiones	Semilla, estado inicial y entradas a 120 Hz; menos de 50 KB por highlight comprimido con zstd (unos 82 KB sin comprimir a 40 s)	Confirmado
Mods	Solo datos: JSON, PNG y SVG, sin DLL ni scripts	Confirmado
PC mínimo	4 núcleos a 3,0 GHz, 8 GB, GTX 1050 o integrada reciente, SSD; 60 fps a 1080p	Confirmado
Multihilo	Tareas de .NET por liga, sin Jobs ni Burst	Confirmado, con ajuste en la API

Los tres ajustes:

- **Multihilo:** `Parallel.ForEachAsync` no existe en el perfil .NET de Unity (.NET Standard 2.1). Se usa `Parallel.ForEach` o `Task.Run` con `Task.WhenAll`, que hacen lo mismo y sí existen (sección 6).
- **Guardado:** copiar el archivo `.sqlite` en bruto con WAL activo puede dejar fuera los cambios que aún están en el archivo `-wal`. La copia se hace con `VACUUM INTO`, que genera un archivo completo y consistente (sección 4).
- **Determinismo:** las operaciones básicas de coma flotante dan lo mismo en cualquier x86-64, pero `Math.Sin`, `Math.Exp` o `Math.Pow` pueden variar en el último bit según la CPU y el compilador. `BallSim` usa sus propias versiones de esas funciones (sección 5).

2. Ensamblados y dependencias

Las dependencias van en una sola dirección: Presentation conoce a todos, Core.Sim y Data solo conocen a Core, y Core no conoce a nadie.



ensamblados · 4 módulos y sus dependencias

Core.Sim y Data no se ven entre sí: el simulador devuelve resultados en memoria y Presentation decide cuándo se escriben en la base de datos.

Ensamblado	Contiene	Referencias permitidas
`GoldenMoments.Core`	Atributos, fórmulas (`A_ef`, GRL, valor de mercado), contratos, reglas de reglamento, competiciones y carrera	Ninguna
`GoldenMoments.Core.Sim`	Motor de simulación del partido, tabla de amenaza de 18 zonas, IA del entrenador rival, generador de semillas	Core
`GoldenMoments.Data`	Esquema, consultas, migraciones, serialización JSON, guardado y compresión	Core, `Microsoft.Data.Sqlite`, `ZstdSharp`
`GoldenMoments.Unity.Presentation`	Pantallas, controladores, `TrajectoryRenderer`, `BallSim` en escena, entradas, audio	Todos, `UnityEngine`

Cómo se garantiza que Core no toque Unity:

- Los `.asmdef` de Core, Core.Sim y Data llevan `noEngineReferences: true`. Si alguien escribe `using UnityEngine;` ahí, no compila.
- Los tres se compilan también fuera de Unity, como bibliotecas `netstandard2.1` en una solución de .NET aparte que comparte los mismos archivos fuente. Es lo que permite las pruebas rápidas de la sección 9.
- `BallSim` tiene dos partes: la integración numérica a 120 Hz vive en Core.Sim (sin Unity) y la representación en escena vive en Presentation. Así las repeticiones se pueden verificar sin abrir el motor.

3. Base de datos SQLite, WAL y migraciones

Cada partida guardada es un archivo `.sqlite` propio, abierto con `Microsoft.Data.Sqlite` en modo WAL: la interfaz lee sin esperar mientras un único hilo escribe los resultados de la jornada.

Configuración al abrir la partida:

- `PRAGMA journal_mode = WAL;`
- `PRAGMA synchronous = NORMAL;` (en WAL es seguro ante cierres del juego; solo un corte de luz puede perder la última transacción).
- `PRAGMA foreign_keys = ON;`
- `PRAGMA wal_autocheckpoint = 4000;` (unos 16 MB de WAL antes de volcar al archivo principal).
- Dos conexiones: una de escritura, propiedad del servicio de guardado, y una de solo lectura para la interfaz.

Un solo escritor. SQLite admite muchos lectores pero solo un escritor a la vez. Los hilos de simulación nunca escriben: devuelven sus resultados en memoria y el servicio de guardado los escribe todos en una sola transacción por jornada. Una jornada mundial (unos 400 partidos de liga) entra en una transacción de menos de 1 s en SSD (estimación, por medir).

Tablas principales (propuesta):

Grupo	Tablas	Crece por temporada
Mundo	`countries`, `competitions`, `clubs`, `stadiums`, `referees`	Casi nada
Jugadores	`players`, `player_attributes`, `player_state`, `hidden_dna (con attr_ceilings como BLOB de 25 bytes, 1 por atributo visible, sin tabla aparte)`, `contracts`	1.600 juveniles nuevos
Temporada	`fixtures`, `matches`, `standings`, `cards_suspensions`, `injuries`	16.280 partidos
Partido	`match_events` (detalle completo solo en partidos del usuario; resumen en el resto)	La tabla más grande
Mercado y carrera	`transfers`, `offers`, `manager_career`, `board_confidence`, `achievements`	Moderado

Grupo	Tablas	Crece por temporada
Historial	`player_season_stats`, `club_season_stats`, `hall_of_fame`	1 fila por jugador y temporada
Bandeja	`inbox_items`	Se purga al caducar

Migraciones:

1. La versión del esquema se guarda en `PRAGMA user_version`.
2. Los scripts viven en Data como recursos incrustados: `001_initial_schema.sql`, `002_add_injury_ceilings.sql`, y así sucesivamente. Nunca se edita un script ya publicado; un cambio es siempre un script nuevo.
3. Al cargar una partida antigua se hace antes una copia de seguridad (`<partida>.pre-003.sqlite.zst`) y después se aplica cada script pendiente en su propia transacción, subiendo `user_version` al terminar cada uno.
4. Si un script falla, se deshace su transacción, se avisa al jugador y la partida se abre desde la copia.
5. Una partida con `user_version` mayor que la del juego (creada con una versión más nueva) no se abre y se muestra un aviso.

Integración en Unity: `Microsoft.Data.Sqlite` necesita la biblioteca nativa `e_sqlite3` de `SQLitePCLRaw` para cada plataforma (Windows x64 como mínimo). Se incluye en `Plugins/` y se añade un `link.xml` para que IL2CPP no elimine sus tipos.

4. Guardado, compresión zstd y Steam Cloud

Un guardado es una instantánea consistente del `.sqlite` comprimida con zstd nivel 3; se hace en segundo plano y el jugador puede seguir usando los menús mientras tanto.

Archivos de una partida:

Archivo	Dónde	Se sincroniza con Steam Cloud
`partida.sqlite` + `-wal` + `-shm`	Carpeta local de trabajo	No: es la copia viva, descomprimida
`partida.gmsave` (manual)	Carpeta de guardados	Sí
`partida.auto1..3.gmsave` (rotatorios)	Carpeta de guardados	Solo el más reciente

Flujo de guardado (Ajuste):

1. Desde la conexión de lectura se ejecuta `VACUUM INTO 'tmp.sqlite'`. Genera un archivo completo que ya incluye lo que estaba en el `-wal`, y de paso lo compacta. Copiar el `.sqlite` en bruto podría dejar fuera las últimas jornadas.
2. `tmp.sqlite` se comprime en streaming con zstd nivel 3 (biblioteca `ZstdSharp`, C# puro, sin binarios nativos).
3. Se escribe a `partida.gmsave.tmp` y después se renombra sobre el anterior. Si el juego se cierra a mitad, el guardado anterior sigue intacto.

Cabecera de `.gmsave` (64 bytes, sin comprimir): firma `GMFC`, versión del formato, versión del juego, `user_version` del esquema, fecha, temporada y jornada, club y mod activo, y un CRC32 del contenido. La pantalla de carga lee solo la cabecera para listar partidas sin descomprimir nada.

Tamaño a 20 temporadas:

Concepto	Valor
Base de datos sin comprimir	400–500 MB (estimación, por medir)
Ratio zstd nivel 3 necesario para quedar por debajo de 150 MB	unas 3,3×
Objetivo	menos de 150 MB por guardado a 20 temporadas
Tiempo de guardado estimado	3–6 s en segundo plano

Un ratio de 4× es habitual en bases de datos con muchos números repetidos, pero hay que medirlo con una partida sintética de 20 temporadas antes de cerrarlo. Si no llega, las palancas son: guardar `match_events` detallado solo para los partidos del usuario, pasar estadísticas antiguas a resúmenes por temporada, o subir a zstd nivel 6.

Steam Cloud: solo se sincroniza el .gmsave comprimido, nunca la carpeta de trabajo. Con hasta 150 MB por archivo, se sincronizan los guardados manuales y el automático más reciente. La cuota por usuario se fija en Steamworks (Auto-Cloud) al medir el guardado real en Acceso Anticipado.

Carga: se descomprime el `.gmsave` a la carpeta de trabajo, se comprueba el CRC32, se aplican migraciones si hace falta (sección 3) y se abre en modo WAL.

5. Determinismo y repeticiones

Con la misma semilla y las mismas entradas, un partido y un highlight dan el mismo resultado en cualquier PC x86-64, siempre que se cumplan las reglas de esta sección.

Semillas:

- `semilla_partida`: se crea al empezar la partida y se guarda en la base de datos.
- `semilla_partido = hash(semilla_partida, id_partido)`.
- Cada jugador y cada subsistema (árbitro, lesiones, IA del entrenador) tiene su propio flujo: `hash(semilla_partido, id)`. Así, añadir una tirada en un subsistema no cambia las de los demás.
- Generador propio en Core (PCG32). No se usa `UnityEngine.Random` ni `System.Random`, cuyo algoritmo puede cambiar entre versiones de .NET.

Reglas del código de simulación (Core.Sim y la parte numérica de `BallSim`):

1. Paso fijo de 1/120 s con acumulador. Nunca se usa `Time.deltaTime` dentro de la simulación. El renderizado interpola entre los dos últimos estados, lo que permite pintar a 60, 144 Hz o cualquier otra frecuencia sin cambiar el resultado.
2. Euler semi-implícito: primero la velocidad, después la posición con la velocidad nueva.
3. **Ajuste:** las funciones trascendentes (`Sin`, `Cos`, `Atan2`, `Exp`, `Pow`) vienen de una clase propia `DetMath` con aproximaciones polinómicas. Las de `System.Math` pueden diferir en el último bit según la CPU y entre Mono (editor) e IL2CPP (juego compilado), y en 2.400 pasos esa diferencia acaba moviendo el balón. `Sqrt`, suma, resta, multiplicación y división sí son exactas por IEEE 754.
4. Nada de recorrer `Dictionary` o `HashSet` en la simulación: su orden no está garantizado. Se recorren listas ordenadas por id.
5. Un partido se simula siempre en un solo hilo. El paralelismo es entre partidos, nunca dentro de uno (sección 6).

Formato de una repetición:

Parte	Contenido	Tamaño en un highlight de 20 s
Cabecera	Versión del juego, `semilla_partido`, id del highlight, minuto	64 B
Estado inicial	Posición y velocidad de los 22 jugadores y del balón, reloj, marcador	1 KB
Entradas	Por paso: puntería en 2 × 16 bits y botones en 8 bits, a 120 Hz	12 KB

Parte	Contenido	Tamaño en un highlight de 20 s
Comprobaciones	Hash del estado cada 120 pasos	80 B
Fotogramas clave	Posiciones cuantizadas de jugadores y balón a 10 Hz	28 KB
Total	Antes de comprimir con zstd	**unos 41 KB**

Los fotogramas clave son un seguro: si una repetición vieja no reproduce igual tras una actualización del juego (lo detectan las comprobaciones), se reproduce interpolando esos fotogramas en lugar de resimular. Se ve igual de bien en cámara normal; solo pierde la posibilidad de cambiar la simulación a mitad de la repetición.

Prueba que lo vigila: un conjunto de repeticiones de referencia se reproduce en cada compilación, en el editor (Mono) y en el juego compilado (IL2CPP), y sus hashes deben coincidir (sección 9).

6. Multihilo de la simulación de fondo

La simulación de fondo reparte las 40 ligas entre los núcleos con tareas de .NET, y el resultado es idéntico se use 1 núcleo o 16.

Ajuste de API: Unity compila contra .NET Standard 2.1, donde no existe `Parallel.ForEachAsync` (llegó en .NET 6). Se usa `Parallel.ForEach` dentro de un `Task.Run`, para que el hilo principal de Unity no se bloquee. Si en el futuro Unity pasa a CoreCLR, el cambio a `ForEachAsync` es local a una clase.

Avance de una jornada:

1. **Instantánea.** Se cargan en memoria, solo para leer, los datos que necesitan los partidos de la jornada: plantillas, estado físico, tácticas y árbitros.
2. **Fase paralela.** `Parallel.ForEach` sobre las ligas, con `MaxDegreeOfParallelism = núcleos - 1` (un núcleo queda libre para Unity). Cada partido escribe su resultado en una posición fija de un array, indexada por id de partido. Nadie toca datos compartidos.
3. **Copas y torneos continentales.** Mismo mecanismo, en una segunda pasada, porque mezclan clubes de varias ligas.
4. **Fase secuencial.** Se aplican, en orden de id, los cambios que afectan a varios clubes: lesiones, sanciones, moral, clasificación, coeficientes y mercado.
5. **Escritura.** El servicio de guardado escribe todo en una transacción (sección 3).
6. **Aviso a la interfaz.** Se envía al hilo principal por una cola que Presentation vacía en cada `Update`.

Como cada partido usa su propia semilla y los resultados se aplican en orden de id, el reparto de hilos no cambia nada. Una prueba simula la misma temporada con 1 y con 8 hilos y compara los hashes de la base de datos.

Tiempos objetivo (con 0,5 ms por partido, el objetivo de «Competiciones y mundo»):

Operación	Partidos	1 hilo	3 hilos (PC mínimo)
Jornada de liga mundial	unos 400	0,2 s	unos 0,07 s
Temporada completa	16.280	unos 8 s	unos 3 s

Cancelación y progreso: todo el avance recibe un `CancellationToken` (salir del juego a mitad de jornada no deja la base de datos a medias, porque la escritura es una sola transacción) y un `IProgress` que alimenta la barra de «Continuar».

7. Mods y Steam Workshop

Un mod es una carpeta de datos (JSON, PNG y SVG) que el juego valida antes de usar; no puede ejecutar código de ningún tipo.

Tipos de mod:

Tipo	Qué cambia	Cuándo se aplica
Mundo	Países, ligas, clubes, jugadores, coeficientes, reputaciones. Las plantillas del mod se cargan tal cual; semilla_partida solo rellena huecos, agentes libres y juveniles	Al crear la partida; se copia dentro del `.sqlite`, así la partida no depende de que el mod siga instalado
Visual	Nombres, escudos, equipaciones, colores	Se puede activar o desactivar en cualquier momento
Parámetros	Valores de las tablas de ajuste del juego, dentro de límites	Al crear la partida; queda registrado en la cabecera del guardado
Traducción	Textos de un idioma nuevo en la carpeta lang/<código>/ con mod.json («Narrativa y presentación», sección 8)	En cualquier momento; no afecta a los logros

Estructura de la carpeta:

```
mi-mod/
  mod.json          (id, nombre, versión, tipo, versión mínima del juego)
  world/*.json      (solo mods de mundo)
  params/*.json     (solo mods de parámetros)
  crests/*.png|svg
  kits/*.png
```

Validación antes de cargar:

- Cada JSON se comprueba contra un esquema publicado (`world.schema.json`, `params.schema.json`). Un error indica archivo, línea y campo.
- Los parámetros solo aceptan claves de una lista blanca y cada una tiene mínimo y máximo. Un mod no puede, por ejemplo, poner a 0 la probabilidad de lesión.
- Rutas: no se aceptan `..`, rutas absolutas ni enlaces simbólicos fuera de la carpeta del mod.
- Imágenes: PNG de 512 × 512 px; SVG opcional. Los SVG se leen con el paquete Vector Graphics de Unity, ignorando `<script>`, enlaces externos e imágenes incrustadas remotas.
- Límite total por mod: 200 MB.

Workshop: la subida y la descarga usan Steamworks.NET, que vive solo en Presentation. Un editor mínimo dentro del juego valida el mod y lo sube; quien prefiera editar los JSON a mano puede hacerlo y usar el mismo validador.

Logros: los mundos del Workshop mantienen los logros, como se decidió en «Carrera del mánager», igual que los mods visuales y de traducción. Los mods de parámetros marcan la partida como modificada y desactivan los logros.

8. Rendimiento y PC mínimo

En el PC mínimo el juego mantiene 60 fps estables a 1080p en los highlights; con mejor hardware se desbloquea hasta 240 Hz o sin límite.

Componente	Mínimo (confirmado)	Recomendado (propuesta)
CPU	4 núcleos a 3,0 GHz (Intel Core i5-7400 / AMD Ryzen 3 1200)	6 núcleos (Intel Core i5-10400 / AMD Ryzen 5 3600)
RAM	8 GB	16 GB
GPU	NVIDIA GTX 1050 2 GB, Intel Iris Xe o AMD Radeon 680M	NVIDIA GTX 1660 / AMD RX 5500 XT
Almacenamiento	SSD recomendado	SSD

Componente	Mínimo (confirmado)	Recomendado (propuesta)
Sistema	Windows 10/11 de 64 bits	Windows 10/11 de 64 bits
Objetivo	60 fps a 1080p	144 fps a 1080p o 60 fps a 1440p

Presupuesto de un fotograma en el PC mínimo (16,6 ms a 60 fps):

Parte	Presupuesto
Simulación del highlight: 2 pasos de 120 Hz, `BallSim` y física	1,5 ms
IA de jugadores de campo y portero	2 ms
Animación	2 ms
Renderizado (CPU)	6 ms
Interfaz y `TrajectoryRenderer`	1,5 ms
Margen	3,6 ms

Memoria: el juego no debe pasar de 3 GB de RAM en el PC mínimo. SQLite usa una caché de 64 MB (`PRAGMA cache_size = -65536`); la interfaz carga listas paginadas, nunca tablas completas.

Frecuencia de pantalla: límite configurable a 30, 60, 120, 144, 165, 240 fps o sin límite, con o sin VSync. La simulación y BallSim siguen en paso fijo de 120 Hz y el renderizado interpola (sección 5), así que jugar a 144 Hz no da ventaja ni cambia el resultado.

Compilación: IL2CPP para Windows x64. Mono solo en el editor.

9. Pruebas e integración continua

Las pruebas de Core, Core.Sim y Data corren con `dotnet test` , sin Unity, en segundos; solo las de Presentation y las de determinismo en el juego compilado necesitan el motor.

Una precisión sobre lo confirmado: las pruebas EditMode de Unity se ejecutan dentro del editor, así que siempre cargan el motor. Lo que permite correrlas en milisegundos es la solución de .NET aparte de la sección 2, que compila los mismos archivos fuente sin Unity. Las mismas pruebas pueden seguir existiendo también como EditMode para quien trabaje desde el editor.

Nivel	Qué prueba	Dónde corre	Cuándo
Unitarias	Fórmulas, reglas, generador de semillas, `DetMath`	`dotnet test`, Linux	Cada push
Simulación	Partidos y temporadas completas; rangos de goles, tarjetas y lesiones de los documentos de simulación y reglamento	`dotnet test`, Linux	Cada push (1.000 partidos) y cada noche (10.000)
Datos	Migraciones de cada versión publicada a la actual, guardado y carga, validación de mods	`dotnet test` con SQLite real	Cada push
Paralelismo	La misma temporada con 1 y 8 hilos da el mismo hash de base de datos	`dotnet test`	Cada noche
Determinismo	Repeticiones de referencia en el editor (Mono) y en el juego compilado (IL2CPP) con el mismo hash	Unity en Windows	Cada noche y antes de cada versión
Rendimiento	Jornada mundial en menos de 200 ms y fotograma en menos de 16,6 ms en una máquina de referencia del PC mínimo	Unity en Windows	Antes de cada versión

Partida de referencia: se genera una partida sintética de 20 temporadas y se guarda en el repositorio como artefacto de pruebas. Sirve para medir el tamaño del guardado (sección 4) y para probar las migraciones sobre datos realistas.

10. Riesgos y preguntas abiertas

Riesgo	Efecto	Mitigación
El ratio de zstd no llega a 4×	Guardados de más de 150 MB	Medir con la partida de referencia; recortar `match_events` y estadísticas antiguas

Riesgo	Efecto	Mitigación
Diferencias de coma flotante entre Mono e IL2CPP	Repeticiones que no reproducen	`DetMath`, prueba nocturna, fotogramas clave como respaldo
`e_sqlite3` eliminado por IL2CPP o ausente	El juego no abre partidas	`link.xml` y prueba de humo en el juego compilado
Guardar en mitad de una escritura	Guardado incompleto	`VACUUM INTO` sobre una lectura consistente y renombrado atómico
Mods con datos extremos	Partidas rotas o desequilibradas	Esquemas, lista blanca y límites por parámetro

Preguntas abiertas:

- [x] ~~¿Los mods de parámetros mantienen los logros? *Confirmado: no; la partida queda marcada como «modificada» en la cabecera y los logros se desactivan. Los mundos y los mods visuales sí los mantienen.*~~
- [x] ~~¿Salen versiones para macOS, Linux o Steam Deck en la 1.0? *Confirmado: solo Windows x64 en 1.0, sin bloquear las otras: Core, Sim y Data ya son multiplataforma y SQLite tiene binarios para todas.*~~
- [ ] ¿Se guarda en la nube la carpeta de trabajo descomprimida? *Propuesta: no, solo los `.gmsave` (sección 4).*
- [ ] Cuota de Steam Cloud por usuario: fijarla cuando se mida el tamaño real del guardado.

11. Módulo de mecánicas en Unity (prototipo 2D, listo para 3D)

El prototipo de mecánicas (límites del campo, portero, tiro y porterías, defensa y entradas) sigue la misma separación de ensamblados de la sección 2. La regla es una: **el núcleo decide y la vista dibuja**. Pasar de 2D a 3D cambia solo la vista.

Ensamblado	Referencias	Contenido
`GoldenMoments.Core.Sim.Match`	Ninguna (`noEngineReferences: true`)	Matemática, balón, porterías, árbitro de líneas, reanudaciones, tiro, portero, defensa
`GoldenMoments.Unity.Presentation.Mechanics`	Core.Sim.Match, Unity.InputSystem	ScriptableObjects de configuración, vistas 2D, entrada, escena de prueba

Ensamblado	Referencias	Contenido
`GoldenMoments.Unity.Presentation.Mechanics.Editor`	La anterior	Tooltips del Inspector a partir de los atributos `[Doc]`

Sistema de coordenadas y paso de 2D a 3D

El núcleo trabaja en metros: X a lo largo del campo (±52,5), Y a lo ancho (±34) y Z la altura. `WorldView` es el único sitio que traduce a la escena:

Modo	Punto en el suelo	Cuerpo del balón
`TopDown2D` (actual)	(x, y, 0)	(x, y + 0,45·z, 0) y algo más grande; la sombra se queda en el suelo
`Perspective3D`	(x, 0, y)	(x, z, y)

Para el 3D se cambian las vistas (`BallView2D` , `PlayerView2D` , `PitchView2D` , `DefenseViews2D`) por sus versiones con mallas y animación, y `WorldView.Mode` a `Perspective3D` . Núcleo, configuración y reglas no se tocan; las pruebas siguen valiendo.

Por qué no se usan Rigidbody ni colliders físicos

- **Determinismo.** PhysX no da el mismo resultado en todas las máquinas ni entre versiones, y la repetición y la simulación rápida (sección 5) necesitan resultados idénticos con la misma semilla.
- **Simulación sin escena.** Los partidos que no se ven se simulan en hilos de fondo (sección 6), donde no existe Unity.
- **Predicción.** La IA del portero y la línea de pase predicen la trayectoria con el mismo integrador que la juega (`BallPhysics.Predict`); con PhysX habría dos físicas distintas.
- **Lo que sí puede usar colliders.** Solo la selección con el ratón (picking) o efectos visuales, nunca la jugada.

Scripts del módulo

Script	Capa	Responsabilidad
`Math/Vec3.cs`	Núcleo	Vector propio, `SimMath` (paso 1/120 s, tablas sin funciones trascendentes) y `DetRandom` (SplitMix64)
`Pitch/PitchSpec.cs`	Núcleo	Medidas del campo y `PlayerBody` (atributos, velocidad, tiempo hasta un punto)
`Ball/BallPhysics.cs`	Núcleo	Integrador del balón y predicción (es el `BallSim` de la sección 5)
`Ball/GoalFrame.cs`	Núcleo	Postes, larguero y red; detección de gol
`Ball/BallWorld.cs`	Núcleo	Paso del balón con porterías, vallas y eventos (gol, palo, fuera)
`Pitch/BoundaryReferee.cs`	Núcleo	Fuera de banda, córner y saque de puerta
`Pitch/RestartController.cs`	Núcleo	Flujo de las reanudaciones y reglas activas
`Shooting/ShotSolver.cs`, `ShotOutcomeOdds.cs`	Núcleo	Tiro sin efecto y probabilidad de palo
`Goalkeeper/SweeperDecision.cs`	Núcleo	Salir o quedarse ante un balón al hueco
`Defense/*.cs`	Núcleo	Exposición del balón, entradas, faltas, cambio de jugador y movimiento del defensor
`Config/MechanicsConfigs.cs`	Presentación	Tres ScriptableObjects: *Campo, balón y porterías*, *Defensa*, *Salidas del portero*
`View/*.cs`	Presentación	Dibujo 2D del campo, red, balón, jugadores e indicadores

Script	Capa	Responsabilidad
`Input/MechanicsInput.cs`	Presentación	Teclado y ratón con el Input System, guardados hasta el siguiente paso
`MechanicsSandbox2D.cs`	Presentación	Escena de prueba: bucle de 120 Hz, modos Defender y Tirar, IA mínima

Bucle de la escena

`MechanicsSandbox2D.Update` acumula el tiempo real y ejecuta pasos fijos de 1/120 s (como mucho 8 por fotograma, para no entrar en espiral). En cada paso: entrada, cambio de jugador, movimiento del defensor, entrada (tackle), IA del poseedor y del portero, paso del balón y reanudaciones. Después, en `LateUpdate`, las vistas copian el estado. Los sistemas guardan una referencia a los ajustes de los assets, así que casi todos los cambios del Inspector se notan en juego al momento; las medidas del campo y de la portería se leen al arrancar y piden reiniciar la escena. R recoloca la jugada.

Pruebas

`S/build/check` es una consola .NET 8 que compila el núcleo sin Unity y reproduce los casos de los documentos (tiros, palos, red, fueras, portero, entradas, cambio de jugador). Debe pasar a `dotnet test` (sección 9) cuando el núcleo entre en el repositorio.